

Project 1 Brief

Numerical Solution of ODE Initial Value Problems

Numerical Analysis of ODEs and PDEs · Weeks 1–4 · Team of 5 · English

1. Task

Choose a stiff or non-stiff ODE initial value problem with a real-world background (see the Problem Pack, `problem_pack`, for five suggested directions). Topics ①–③ are the standard deterministic-ODE pathway. Topics ④–⑤ are extension pathways whose method and analysis requirements are modified in the Problem Pack; follow the pathway-specific requirements there. Build a small numerical library around it, and answer a mathematical question about it using numerical experiments.

You are not asked to reproduce a textbook example. You are asked to **act like a consultant**: given a phenomenon (a bouncing satellite, a firing neuron, a reacting chemical system, an option price), produce a reliable, validated numerical answer, and explain — *quantitatively* — why your methods can be trusted.

2. Examples

The following works may appear in your report and code. Every work is checked against a reference solution **you construct yourself** — we deliberately do not hand you benchmark numbers, because learning to judge “is this right?” is part of the project.

1. Three methods

- Explicit Euler (order 1)
- RK4 (order 4)
- One implicit method (implicit Euler or trapezoidal / Crank–Nicolson for ODEs), with the resulting nonlinear system solved by Newton’s iteration

2. Stability analysis

- Absolute-stability region of each of your three methods (derived and plotted)
- For your system, the relevant eigenvalue range (linearised where necessary), and the resulting practical step-size restriction for the explicit methods
- State that an $h\lambda$ overlay is a *local frozen-Jacobian diagnostic*, not a complete nonlinear stability proof; test its prediction with a step-size or perturbation experiment

3. Adaptive step-size control (basic version is fine)

- e.g. step-doubling with local-error estimation, or an embedded pair
- Show it working: the step size should actually change, and the error should land near the tolerance you set

4. Validation

- Build a high-accuracy reference solution **yourself** (see below)

- Produce an error-vs-step-size log-log plot with the observed order of convergence for at least two methods

5. **Report** (≥ 8 pages, LaTeX; submit the PDF)

- Structure: Background $\rightarrow$ Model $\rightarrow$ Methods $\rightarrow$ Stability $\rightarrow$ Numerical experiments $\rightarrow$ Conclusions. See the LaTeX template in 06_Templates/.

6. **Code**

- A zip file with a README explaining how to run it
- Everyone runs `run_all.py` to reproduce all figures

7. **Slides** (PDF, one set for your presentation wave)

- The deck tells the full story arc (background $\rightarrow$ model $\rightarrow$ methods $\rightarrow$ experiments $\rightarrow$ conclusions); each slide has a takeaway, not just bullet points
- 10 min presentation + 3 min Q&A

Deadline: the Monday after the final tutorial, 23:59. Late policy: 8-hour grace period, then -10% per hour (see the Syllabus).

How to verify without a given answer. There is no handed-out benchmark value in this course. Instead, build your own ground truth with at least **two** independent checks:

1. **Independent oracle:** `scipy.integrate.solve_ivp` with a very tight tolerance (`rtol=1e-12`, an implicit method such as BDF or Radau) gives a reference trajectory. This is a *tool you choose*, not an answer we give you.
2. **Known cases:** where your system has an analytic or exactly-solvable special case (a linearised version, a reduced 1-D model, a conserved quantity like energy or mass), check against it.
3. **Self-consistency:** two of your own methods with very different orders must agree on the fine-mesh limit; the convergence-order plot itself is a check (a slope that matches theory on a known case gives confidence).

In the report, state *which* checks you used, *what* tolerance/convergence criterion you accepted, and *where* your numbers come from. That chain of evidence is what we grade — not the numbers themselves.

3. Examples of tiered tasks

Marked with stars. ★ = basic (must complete), ★★ = advanced (aim to complete), ★★★ = challenge (for strong teams).

- ★ Implement Euler, RK4, and one implicit method.
- ★ Validate order on a problem; log-log error plot.
- ★ Basic adaptive step-size control with a stated tolerance.
- ★★ Quantitative stability analysis.

- ★★ Quantitative stiffness discussion: eigenvalue spread (or stiffness ratio) of your linearised system.
- ★★ For the mechanics topic: exploit a conserved quantity (energy, angular momentum) as a *diagnostic* — show which integrator respects it and why.
- ★★★ Implement suitable advanced methods for a particular problem, e.g. positivity-preserving high-order methods.
- ★★★ Develop convergence theory for non-standard methods or problems.

4. Suggested topics (details in the Problem Pack)

#	Topic	Type	Core numerical lesson
①	Celestial mechanics: restricted three-body	conservative	Jacobi-invariant diagnostics, accuracy vs. stability, adaptivity; optional structure-preserving integration
②	Hodgkin–Huxley neuron model	moderately stiff	state-dependent fast–slow time scales, accuracy and stability limits, event-aware adaptivity
③	Chemical kinetics: Robertson problem	stiff, benchmark	conservation-manifold eigenanalysis, stiffness ratio grows to $\sim 1.6 \times 10^5$ at $t = 40$, positivity and implicit robustness
④	Financial SDE: geometric Brownian motion	SDE (extension)	Euler–Maruyama / Milstein, strong vs. weak convergence
⑤	Matrix differential equations: polar-factor flow	matrix ODE (extension)	exact matrix solution, Lyapunov diagnostics, neutral and contracting modes

For topics ④ and ⑤, follow the pathway-specific requirements in the Problem Pack rather than mechanically applying every scalar/deterministic item in Section 2. You may also propose your own topic.

5. Example timeline and milestones

Week	Milestone	Deliverable due
1	Topic chosen, environment set up, Euler runs	Weekly-Milestone entry (topic choice + first run)
2	RK4 + implicit method + Newton working	Weekly-Milestone entry
3	Voluntary first-wave presentation (Seminar 1)	slides (PDF) + work-in-progress draft
4	Second-wave presentation (Seminar 2)	team report PDF (AI log + ICS appendices) + code + slides

Why present in the first wave? It is voluntary but strongly encouraged: you present the work-in-progress, get the teacher’s and peers’ feedback *now*, catch modelling or implementation mistakes while they are still cheap to fix, and have a full week to improve before the final report. The presentation is assessed in the wave in which you present (each team presents once), and presenting work-in-progress is expected and acceptable: the report and code remain revisable until the common deadline, so a first-wave presentation does not lock in your final grade.

6. Grading (this project)

Component	Weight
Mathematical Soundness (methods, analysis)	15%
Depth and Novelty (topic, theory, algorithm)	15%
Implementation: Robust and Reproducible	15%
Numerical experiments and visualization	15%
Writing and presentation	15%
Individual Contribution (ICS)	25%

Detailed descriptors: the course-level rubric (`../Numerical_PDEs_Assessment_Rubrics.pdf`), used for every team’s single presentation, together with the report and code.

7. What we will not accept

- A report that describes methods but shows no experiments
- Code that cannot be run by the instructor on a machine with dependencies installed
- A single team member who cannot explain the project (checked by cross-questioning in the Seminar Q&A)

Scored criterion (Dimension C, Implementation): the **core method must be correctly applied** — you may implement it directly or call a well-justified library routine (`numpy/scipy` are fine, including `scipy.integrate.solve_ivp`), but the choice of what to implement yourself vs. call must be **explicit and justified**, and you must be able to explain and modify the code. Vectorise where appropriate (array operations, not for loops over individual elements). Correct application and justified choices are what this dimension rewards.

Ask yourself, after every experiment: “If I were the instructor, would this convince me?” If the answer is no, keep working.

8. Using AI effectively

AI is welcome in this course, but it changes *what* you must do, not *whether* you can succeed. Two habits make the difference between learning and copying:

1. **Plan first, then let AI help step by step.** Before asking an AI tool for code, write down the plan yourself: the method, the algorithm, the expected order of convergence, the sanity checks. Then use AI for individual steps (“implement this one function”, “debug this error”), and verify each step against your plan. If the AI output contradicts the plan, treat the discrepancy as a testable question. Derive a special case, construct a unit test, or consult an authoritative source before deciding which is wrong.
2. **Interrogate the output.** For every AI-generated result, ask: *is the model correct? do we need a numerical method at all? which method is appropriate, and why does it work?* If you cannot answer these four questions about your own report, you are not ready for the viva.

The AI Transparency Log (appendix of the report template) records what you changed in AI output and why — that log *is* your defence, not your confession.